

TASK 3 : SQL QUERIES

DONE BY: Dharshini K

DATE: 29/04/26

BASIC TO INTERMEDIATE:

1) List all customers and the total number of orders they have placed. Show only customers with more than 5 orders. Sort by total orders descending. The total sales amount per customer by joining customers, orders, and order_details.

ANSWER:

```
SELECT c.customerId, c.companyname, COUNT(o.orderID) AS total_orders
FROM Customers c JOIN Orders o ON c.customerID = o.customerID
GROUP BY c.customerID HAVING COUNT(o.orderID) > 5
ORDER BY COUNT(o.order ID) DESC;
```

OUTPUT:

```
demo=# SELECT c.customerId, c.companyname, COUNT(o.orderID) AS total_orders FROM Customers c JOIN Orders
o ON c.customerID = o.customerID GROUP BY c.customerID HAVING COUNT(o.orderID) > 5 ORDER BY COUNT(o.order
ID) DESC;
```

customerid	companyname	total_orders
SAVEA	Save-a-lot Markets	31
ERNSH	Ernst Handel	30
QUICK	QUICK-Stop	28
FOLKO	Folk och få HB	19
HUNGO	Hungry Owl All-Night Grocers	19
HILAA	HILARION-Abastos	18
BERGS	Berglunds snabbköp	18
RATTC	Rattlesnake Canyon Grocery	18
BONAP	Bon app'	17
WARTH	Wartian Herkku	15

2) Retrieve the total sales amount per customer by joining customers, orders, and order_details. Show only customers whose total sales exceed 10,000. Sort by total sales descending.

ANSWER:

```
SELECT c.customerid, c.companyname, SUM(od.unitprice * od.quantity - (1 -
od.discount)) AS total_sales
FROM Customers c JOIN Orders o ON c.customerid = o.customerid
JOIN order_details od ON o.orderid = od.orderid
GROUP BY c.customerID HAVING total_sales > 10000
ORDER BY total_sales DESC ;
```

OUTPUT:

```
demo=# SELECT c.customerid, c.companyname, SUM(od.unitprice * od.quantity - (1 - od.discount) * od.quantity) AS total_sales FROM Customers c JOIN Orders o ON c.customerid = o.customerid JOIN order_details od ON o.orderid = od.orderid GROUP BY c.customerid HAVING SUM(od.unitprice * od.quantity - (1 - od.discount) * od.quantity) > 100000 ORDER BY total_sales DESC ;
```

customerid	companyname	total_sales
QUICK	QUICK-Stop	117403.34000008868
SAVEA	Save-a-lot Markets	115566.99000013931
ERNSH	Ernst Handel	113141.63000012591
HUNGO	Hungry Owl All-Night Grocers	57268.64000006333
RATTC	Rattlesnake Canyon Grocery	52177.39000001699
HANAR	Hanari Carnes	34071.300000035764
FOLKO	Folk och fä HB	32514.400000053643
MEREP	Mère Paillarde	32173.75000002757
KOENE	Königlich Essen	31708.3000000529
QUEEN	Queen Cozinha	30189.700000038738

3) Get the number of products per category. Show only categories having more than 10 products. Sort by product count descending.

ANSWER:

```
SELECT c.categoryid, c.categoryname, COUNT(p.productid) AS product_count FROM categories c JOIN products p ON c.categoryid = p.categoryid GROUP BY c.categoryid HAVING COUNT(p.productid) > 10 ORDER BY product_count DESC;
```

OUTPUT:

```
demo=# SELECT c.categoryid, c.categoryname, COUNT(p.productid) AS product_count FROM categories c JOIN products p ON c.categoryid = p.categoryid GROUP BY c.categoryid HAVING COUNT(p.productid) > 10 ORDER BY product_count DESC ;
```

categoryid	categoryname	product_count
3	Confections	13
2	Condiments	12
1	Beverages	12
8	Seafood	12

(4 rows)

4) Display the total quantity sold per product. Include only products where total quantity sold is greater than 100. Sort by quantity descending.

ANSWER:

```
SELECT p.productid, p.productname, SUM(od.quantity) AS total_sales FROM products p JOIN order_Details od ON p.productid = od.productid GROUP BY p.productid HAVING SUM(od.quantity) > 100 ORDER BY total_sales DESC;
```

OUTPUT:

```
[demo=# SELECT p.productid, p.productname , SUM(od.quantity) AS total_sales FROM products p JOIN order_Details od ON p.productid = od.productid GROUP BY p.productid HAVING SUM(od.quantity) > 100 ORDER BY total_sales DESC;
```

productid	productname	total_sales
60	Camembert Pierrot	1577
59	Raclette Courdavault	1496
31	Gorgonzola Telino	1397
56	Gnocchi di nonna Alice	1263
16	Pavlova	1158
75	Rhönbräu Klosterbier	1155
24	Guaraná Fantástica	1125
40	Boston Crab Meat	1103

5) Find the total number of orders handled by each employee. Show only employees who handled more than 20 orders. Sort by order count descending.

ANSWER:

```
SELECT e.employeeid, CONCAT(e.firstname, ' ', e.lastname) AS name, COUNT(o.orderID) FROM employees e JOIN orders o ON e.employeeid = o.employeeid GROUP BY e.employeeid HAVING COUNT(o.orderid) > 20 ORDER BY COUNT(o.orderid) DESC;
```

OUTPUT:

```
[demo=# SELECT e.employeeid, CONCAT(e.firstname, ' ', e.lastname) AS name, COUNT(o.orderID) FROM employees e JOIN orders o ON e.employeeid = o.employeeid GROUP BY e.employeeid HAVING COUNT(o.orderid) > 20 ORDER BY COUNT(o.orderid) DESC;
```

employeeid	name	count
4	Margaret Peacock	156
3	Janet Leverling	127
1	Nancy Davolio	123
8	Laura Callahan	104
2	Andrew Fuller	96
7	Robert King	72

INTERMEDIATE

6) Retrieve the total sales per category by joining categories, products and order_details. Show only categories with total_sales above 50,000. Sort by total sales descending.

ANSWER:

```
SELECT c.categoryid, c.categoryname, SUM(od.unitprice * od.quantity - (1 - od.discount)) AS total_sales FROM categories c JOIN products p ON c.categoryid = p.categoryid JOIN order_details od ON p.productid = od.productid GROUP BY c.categoryid HAVING SUM(od.unitprice * od.quantity - (1 - od.discount)) > 50000 ORDER BY total_sales DESC;
```

OUTPUT:

```
[demo=# SELECT c.categoryid, c.categoryname, SUM(od.unitprice * od.quantity - (1 - od.discount)) AS total_
sales FROM categories c JOIN products p ON c.categoryid = p.categoryid JOIN order_details od ON p.product
id = od.productid GROUP BY c.categoryid HAVING SUM(od.unitprice * od.quantity - (1 - od.discount)) > 5000
0 ORDER BY total_sales DESC;
categoryid | categoryname | total_sales
-----+-----+-----
1 | Beverages | 286147.9500003874
4 | Dairy Products | 250984.06000030413
6 | Meat/Poultry | 178026.95000012888
3 | Confections | 176784.12000028818
8 | Seafood | 141312.97000027652
2 | Condiments | 113490.12000019105
7 | Produce | 105138.78000009843
5 | Grains/Cereals | 100539.68000013121
(8 rows)
```

7) List suppliers and the number of products they supply. Show only suppliers who supply more than 5 products. Sort by product count descending.

ANSWER:

a) Products_count > 5

```
SELECT s.supplierid, s.companyname, COUNT(p.productid) AS products_count
FROM suppliers s
JOIN products p ON s.supplierid = p.supplierid
GROUP BY s.supplierid HAVING COUNT(p.productid) >= 5
ORDER BY products_count DESC;
```

```
[demo=# SELECT s.supplierid, s.companyname, COUNT(p.productid) AS products_count FROM suppliers s JOIN pro
ducts p ON s.supplierid = p.supplierid GROUP BY s.supplierid HAVING COUNT(p.productid) > 5 ORDER BY produ
cts_count DESC;
supplierid | companyname | products_count
-----+-----+-----
(0 rows)
```

b) Products_count >= 5

```
SELECT s.supplierid, s.companyname, COUNT(p.productid) AS products_count
FROM suppliers s
JOIN products p ON s.supplierid = p.supplierid
GROUP BY s.supplierid HAVING COUNT(p.productid) >= 5
ORDER BY products_count DESC;
```

```
[demo=# SELECT s.supplierid, s.companyname, COUNT(p.productid) AS products_count FROM suppliers s JOIN pro
ducts p ON s.supplierid = p.supplierid GROUP BY s.supplierid HAVING COUNT(p.productid) >= 5 ORDER BY prod
ucts_count DESC;
supplierid | companyname | products_count
-----+-----+-----
7 | Pavlova, Ltd. | 5
12 | Plutzer Lebensmittelgroßmärkte AG | 5
(2 rows)
demo=#
```

8) Get the average unit price per category. Show only categories where the average price is above 30. Sort by average price descending.

ANSWER:

```
SELECT c.categoryid, c.categoryname, AVG(od.unitprice) as Avg_Unit_price
FROM categories c JOIN products p ON c.categoryid = p.categoryid
JOIN order_details od ON p.productid = od.productid
GROUP BY c.categoryid HAVING AVG(od.unitprice) > 30
ORDER BY Avg_Unit_price DESC;
```

OUTPUT:

```
[demo=# SELECT c.categoryid, c.categoryname, AVG(od.unitprice) as Avg_Unit_price FROM categories c JOIN pr
oducts p ON c.categoryid = p.categoryid JOIN order_details od ON p.productid = od.productid GROUP BY c.ca
tegoryid HAVING AVG(od.unitprice) > 30 ORDER BY Avg_Unit_price DESC;
categoryid | categoryname | avg_unit_price
-----+-----+-----
          6 | Meat/Poultry | 42.8747398843930636
          7 | Produce      | 35.1944852941176471
(2 rows)
demo=#
```

9) Display the total revenue generated per employee (orders + order_details). Show only employees generating more than 20,000 in revenue. Sort by revenue descending.

ANSWER:

```
SELECT e.employeeid, CONCAT(e.firstname, ' ', e.lastname) as name,
SUM(od.unitprice * od.quantity - (1 - od.discount)) AS total_sales
FROM employees e JOIN orders o ON e.employeeid = o.employeeid
JOIN order_details od ON o.orderid = od.orderid
GROUP BY e.employeeid HAVING SUM(od.unitprice * od.quantity - (1 -
od.discount)) > 20000
ORDER BY total_sales DESC;
```

OUTPUT:

```
[demo=# SELECT e.employeeid, CONCAT(e.firstname, ' ', e.lastname) as name, SUM(od.unitprice * od.quantity -
(1 - od.discount)) AS total_sales FROM employees e JOIN orders o ON e.employeeid = o.employeeid JOIN orde
r_details od ON o.orderid = od.orderid GROUP BY e.employeeid HAVING SUM(od.unitprice * od.quantity - (1 -
od.discount)) > 20000 ORDER BY total_sales DESC;
employeeid | name | total_sales
-----+-----+-----
          4 | Margaret Peacock | 249793.20000037245
          3 | Janet Leverling | 212746.25000022273
          1 | Nancy Davolio | 201815.70000022562
          2 | Andrew Fuller | 177518.81000018705
          7 | Robert King | 141132.9400001967
          8 | Laura Callahan | 133055.63000021753
          9 | Anne Dodsworth | 82864.30000014229
          6 | Michael Suyama | 78039.25000013634
          5 | Steven Buchanan | 75458.55000010507
(9 rows)
```


10) Retrieve the number of orders shipped to each country. Show only countries with more than 10 orders. Sort by order count descending.

ANSWER:

```
SELECT shipcountry, COUNT(*) AS orders_count FROM Orders
GROUP BY shipcountry HAVING COUNT(*) > 10
ORDER BY orders_count DESC;
```

OUTPUT:

```
demo=# SELECT shipcountry, COUNT(*) AS orders_count FROM Orders GROUP BY shipcountry HAVING COUNT(*) > 10]
ORDER BY orders_count DESC;
shipcountry | orders_count
-----+-----
USA         |         122
Germany     |         122
Brazil      |          83
France      |          77
UK          |          56
Venezuela   |          46
Austria     |          40
Sweden      |          37
Canada      |          30
Italy       |          28
Mexico      |          28
Spain       |          23
Finland     |          22
Ireland     |          19
Belgium     |          19
Denmark     |          18
Switzerland |          18
Argentina   |          16
Portugal    |          13
(19 rows)
```

ADVANCED

11) Find customers and the average order value (orders + order_details). Show only customers with average order value greater than 500. Sort by average descending.

ANSWER:

```
SELECT c.customerid, c.companyname, AVG(od.unitprice * od.quantity - (1 -
od.discount)) as avg_orders
FROM customers c JOIN orders o ON c.customerid = o.customerid
JOIN order_Details od ON o.orderID = od.orderID
GROUP BY c.customerid HAVING AVG(od.unitprice * od.quantity - (1 -
od.discount)) > 500
ORDER BY avg_orders DESC;
```

OUTPUT:

```
demo=# SELECT c.customerid, c.companyname, AVG(od.unitprice * od.quantity - (1 - od.discount)) as avg_orders FROM customers c JOIN orders o ON c.customerid = o.customerid JOIN order_Details od ON o.orderID = od.orderID GROUP BY c.customerid HAVING AVG(od.unitprice * od.quantity - (1 - od.discount)) > 500 ORDER BY avg_orders DESC;
```

customerid	companyname	avg_orders
QUICK	QUICK-Stop	1365.1551162801009
SIMOB	Simons bistro	1208.3700000008444
PICCO	Piccolo und mehr	1140.808695653081
ERNSH	Ernst Handel	1109.231666667901
HANAR	Hanari Carnes	1064.7281250011176
HUNGO	Hungry Owl All-Night Grocers	1041.2480000011515

12) Get the top-selling products per category (by total quantity sold).

Show only products with total quantity sold above 200. Sort within category by quantity descending.

ANSWER:

```
SELECT c.categoryname, p.productid, p.productname, SUM(od.quantity) AS total_quantity
FROM products p JOIN order_details od ON p.productid = od.productid
JOIN categories c ON p.categoryid = c.categoryid
GROUP BY c.categoryname, p.productid, p.productname
HAVING SUM(od.quantity) > 200
ORDER BY c.categoryname, total_quantity DESC;
```

OUTPUT:

```
demo=# SELECT c.categoryname, p.productid, p.productname, SUM(od.quantity) AS total_quantity FROM products p JOIN order_details od ON p.productid = od.productid JOIN categories c ON p.categoryid = c.categoryid GROUP BY c.categoryname, p.productid, p.productname HAVING SUM(od.quantity) > 200 ORDER BY c.categoryname, total_quantity DESC;
```

categoryname	productid	productname	total_quantity
Beverages	75	Rhönbräu Klosterbier	1155
Beverages	24	Guaraná Fantástica	1125
Beverages	2	Chang	1057
Beverages	76	Lakkalikööri	981
Beverages	35	Steeleye Stout	883
Beverages	1	Chai	828
Beverages	70	Outback Lager	817
Beverages	39	Chartreuse verte	793
Beverages	38	Côte de Blaye	623
Beverages	43	Ipoh Coffee	580
Beverages	34	Sasquatch Ale	506
Condiments	77	Original Frankfurter grüne Soße	491

13) Retrieve the total discount given per product (order_details). Show only products where total discount exceeds 1,000. Sort by discount descending.

ANSWER:

```
SELECT p.productId, p.productName, SUM(od.discount * od.unitprice * od.quantity) AS Total_Discount
FROM products p JOIN order_details od ON p.productId = od.productId
GROUP BY p.productId
HAVING SUM(od.discount * od.unitprice * od.quantity) > 1000
```

ORDER BY total_discount DESC;

OUTPUT:

```
[demo=# SELECT p.productID, p.productName, SUM(od.discount * od.unitprice * od.quantity) AS Total_Discount]
FROM products p JOIN order_details od ON p.productID = od.productID GROUP BY p.productID HAVING SUM(od.d
iscount * od.unitprice * od.quantity) > 1000 ORDER BY total_discount DESC;
```

productid	productname	total_discount
38	Côte de Blaye	8587.465096551552
29	Thüringer Rostbratwurst	7367.728025485949
59	Raclette Courdavault	5140.300090570003
60	Camembert Pierrot	3460.5200469806787
51	Manjimup Dried Apples	2922.9500460036097
18	Carnarvon Tigers	2815.625036600977
17	Alice Mutton	2783.820019108057
62	Tarte au sucre	2592.930045986175

14) List employees and the number of unique customers they handled.

Show only employees who handled more than 15 unique customers. Sort by count descending.

ANSWER:

```
SELECT e.employeeID, CONCAT(e.firstname, ' ', e.lastname) AS name,
COUNT(DISTINCT o.customerID) AS count_unique_customers
FROM Employees e JOIN orders o ON e.employeeid = o.employeeid
GROUP BY e.employeeid HAVING COUNT(DISTINCT o.customerID) > 15
ORDER BY count_unique_customers DESC;
```

OUTPUT:

```
[demo=# SELECT e.employeeID, CONCAT(e.firstname, ' ', e.lastname) AS name, COUNT(DISTINCT o.customerID) AS c]
ount_unique_customers FROM Employees e JOIN orders o ON e.employeeid = o.employeeid GROUP BY e.employeeid
HAVING COUNT(DISTINCT o.customerID) > 15 ORDER BY count_unique_customers DESC;
```

employeeid	name	count_unique_customers
4	Margaret Peacock	75
1	Nancy Davolio	65
3	Janet Leverling	63
2	Andrew Fuller	59
8	Laura Callahan	56
7	Robert King	45
6	Michael Suyama	43
9	Anne Dodsworth	29
5	Steven Buchanan	29

15) Find the monthly total sales (year + month) using orders and order_details. Show only months where total sales exceed 30,000. Sort by year and month ascending.

ANSWER:

```
SELECT EXTRACT(YEAR FROM o.orderdate) AS year, EXTRACT(MONTH FROM
o.orderdate) AS month, SUM(od.unitprice * od.quantity * (1 - od.discount)) AS
total_sales
FROM orders o JOIN order_details od ON o.orderid = od.orderid
GROUP BY EXTRACT(YEAR FROM o.orderdate), EXTRACT(MONTH FROM
o.orderdate)
```


HAVING SUM(od.unitprice * od.quantity * (1 - od.discount)) > 30000
ORDER BY year ASC, month ASC;

OUTPUT:

year	month	total_sales
1996	10	37515.72491448447
1996	11	45600.04494312592
1996	12	45239.629965431996
1997	1	61258.0699304849
1997	2	38483.63494294286
1997	3	38547.21996988951
1997	4	53032.95242693588
1997	5	53781.2899279859
1997	6	36362.80244842321
1997	7	51020.85745272498
1997	8	47287.669948245915
1997	9	55629.242456566535

TRANSACTIONS AND SP

16) Create a stored procedure to insert a new customer. Use a transaction so that if any required value is missing, the insert is rolled back.

ANSWER:

CREATE TABLE Customers(CustomerID INT PRIMARY KEY, name VARCHAR(50) NOT NULL, MobileNo CHAR(10) ~ '^[0-9]{10}\$', Email VARCHAR(50) NOT NULL);

```
CREATE OR REPLACE PROCEDURE add_customer(
    customerid INT,
    Customername VARCHAR,
    mobileno CHAR(10),
    email VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    IF p_customerid IS NULL OR p_name IS NULL OR p_email IS NULL THEN
        RAISE EXCEPTION 'CustomerID, Name, and Email are required';
    END IF;
```

```

IF p_mobilenumber IS NOT NULL AND p_mobilenumber !~ '^[0-9]{10}$' THEN
    RAISE EXCEPTION 'Invalid Mobile number';
END IF;

```

```

INSERT INTO Customers(CustomerID, name, MobileNo, Email)
VALUES (customerid, Customername, mobilenumber, email);
COMMIT;

```

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Error occurred';
END;
$$;

```

```
CALL add_customer(1,'Dharshini','1234567890','dharshini@mail.com');
```

OUTPUT:

```

10
11 IF p_customerid IS NULL OR p_name IS NULL OR p_email IS NULL THEN
12     RAISE EXCEPTION 'CustomerID, Name, and Email are required';
13 END IF;
14
15 IF p_mobilenumber IS NOT NULL AND p_mobilenumber !~ '^[0-9]{10}$' THEN
16     RAISE EXCEPTION 'Invalid Mobile number';
17 END IF;
18
19
20 INSERT INTO Customers(CustomerID, name, MobileNo, Email)
21 VALUES (customerid, Customername, mobilenumber, email);
22
23
24 COMMIT;
25
26 EXCEPTION
27     WHEN OTHERS THEN
28         ROLLBACK;
29         RAISE NOTICE 'Error occurred';

```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 1 secs 606 msec.

2) Create a stored procedure to place a new order for an existing customer with one product. Insert into orders and order_details in a single transaction.

ANSWER:

```

CREATE OR REPLACE PROCEDURE order_procedure(
    newOrderid INT,
    newCustomerid VARCHAR,
    newEmployeeid INT,

```

```

        newProductid INT,
        newQuantity INT,
        newDiscount FLOAT
    )
LANGUAGE plpgsql
AS $$
DECLARE
    price NUMERIC;
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM customers WHERE customerid = newCustomerid
    ) THEN
        RAISE EXCEPTION 'Customer does not exist';
    END IF;

    SELECT unitprice INTO price
    FROM products
    WHERE productid = newProductid;

    IF price IS NULL THEN
        RAISE EXCEPTION 'Product does not exist';
    END IF;

    INSERT INTO orders(
        orderid, customerid, employeeid, orderdate
    )
    VALUES (
        newOrderid, newCustomerid, newEmployeeid, CURRENT_DATE
    );

    INSERT INTO order_details(
        orderid, productid, unitprice, quantity, discount
    )
    VALUES (
        newOrderid, newProductid, price, newQuantity, newDiscount
    );

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN

```

```

ROLLBACK;
RAISE NOTICE 'Rolled back due to Transaction failure';
END;
$$;

```

OUTPUT:

```

Query    Query History
25      RAISE EXCEPTION 'Product does not exist';
26      END IF;
27
28      INSERT INTO orders(
29          orderid, customerid, employeeid, orderdate
30      )
31      VALUES (
32          newOrderId, newCustomerId, newEmployeeid, CURRENT_DATE
33      );
34
35      INSERT INTO order_details(
36          orderid, productid, unitprice, quantity, discount
37      )
38      VALUES (
39          newOrderId, newProductid, price, newQuantity, newDiscount
40      );
41
42      COMMIT;
43
Data Output  Messages  Notifications
CREATE PROCEDURE
Query returned successfully in 2 secs 351 msec.

```

3) Create a stored procedure to update product stock after an order is placed. If stock is not enough, rollback the transaction.

ANSWER:

```

CREATE OR REPLACE PROCEDURE placeOrder(
    newOrderId INT,
    newCustomerId VARCHAR,
    newEmployeeid INT,
    newProductid INT,
    newQuantity INT,
    newDiscount FLOAT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_price NUMERIC;
    v_stock INT;
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM customers WHERE customerid = newCustomerId

```

```

) THEN
    RAISE EXCEPTION 'Customer does not exist';
END IF;

SELECT unitprice, unitsinstock
INTO v_price, v_stock
FROM products
WHERE productid = newProductId
FOR UPDATE;

IF v_price IS NULL THEN
    RAISE EXCEPTION 'Product does not exist';
END IF;

IF v_stock < newQuantity THEN
    RAISE EXCEPTION 'Insufficient stock';
END IF;

INSERT INTO orders(orderid, customerid, employeeid, orderdate)
VALUES (newOrderId, newCustomerId, newEmployeeId, CURRENT_DATE);

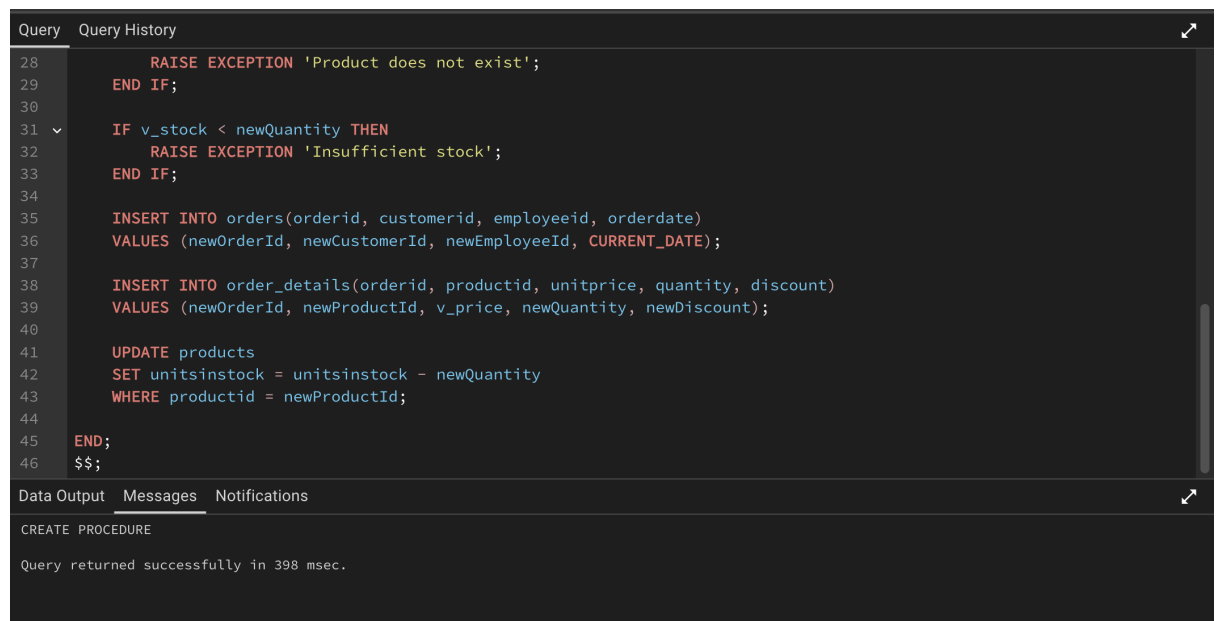
INSERT INTO order_details(orderid, productid, unitprice, quantity, discount)
VALUES (newOrderId, newProductId, v_price, newQuantity, newDiscount);

UPDATE products
SET unitsinstock = unitsinstock - newQuantity
WHERE productid = newProductId;
COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Rolled back due to Transaction failure';
END;
$$;

```


OUTPUT:



The screenshot shows a SQL IDE interface with a dark theme. The top bar has tabs for 'Query' and 'Query History'. The main editor displays a PL/SQL block with line numbers 28 to 46. The code includes exception handling for 'Product does not exist' and 'Insufficient stock', followed by INSERT and UPDATE statements. The bottom bar has tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing the message 'Query returned successfully in 398 msec.'

```
28      RAISE EXCEPTION 'Product does not exist';
29    END IF;
30
31    IF v_stock < newQuantity THEN
32      RAISE EXCEPTION 'Insufficient stock';
33    END IF;
34
35    INSERT INTO orders(orderid, customerid, employeeid, orderdate)
36    VALUES (newOrderId, newCustomerId, newEmployeeId, CURRENT_DATE);
37
38    INSERT INTO order_details(orderid, productid, unitprice, quantity, discount)
39    VALUES (newOrderId, newProductId, v_price, newQuantity, newDiscount);
40
41    UPDATE products
42    SET unitsinstock = unitsinstock - newQuantity
43    WHERE productid = newProductId;
44
45  END;
46  $$;
```

Query returned successfully in 398 msec.

4) Create a stored procedure to cancel an order. Delete records from order_details first, then from orders, using a transaction.

ANSWER:

```
CREATE OR REPLACE PROCEDURE cancelOrder(
  newOrderId INT
)
LANGUAGE plpgsql
AS $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM orders WHERE orderid = newOrderId
  ) THEN
    RAISE EXCEPTION 'Order does not exist';
  END IF;

  DELETE FROM order_details
  WHERE orderid = newOrderId;

  DELETE FROM orders
  WHERE orderid = newOrderId;
  COMMIT;

  EXCEPTION:
    IF OTHERS THEN
```

```

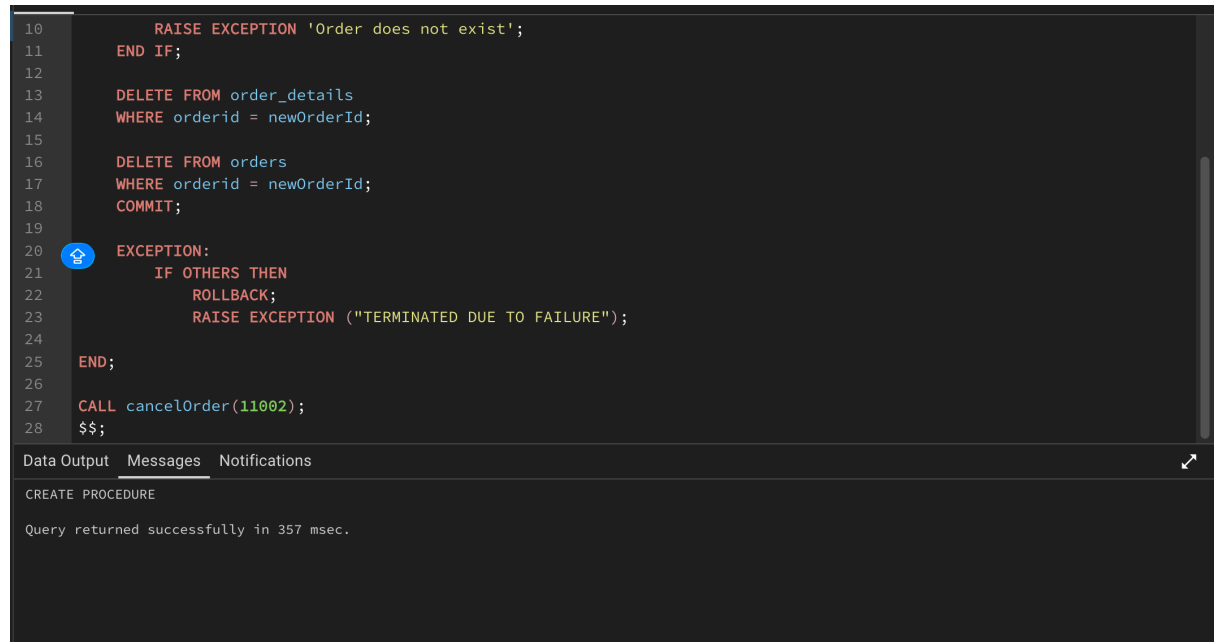
        ROLLBACK;
        RAISE EXCEPTION ("TERMINATED DUE TO FAILURE");

END;

CALL cancelOrder(11002);
$$;

```

OUTPUT:



```

10      RAISE EXCEPTION 'Order does not exist';
11      END IF;
12
13      DELETE FROM order_details
14      WHERE orderid = newOrderId;
15
16      DELETE FROM orders
17      WHERE orderid = newOrderId;
18      COMMIT;
19
20      EXCEPTION:
21      IF OTHERS THEN
22          ROLLBACK;
23          RAISE EXCEPTION ("TERMINATED DUE TO FAILURE");
24
25      END;
26
27      CALL cancelOrder(11002);
28      $$;

```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 357 msec.

5) Create a stored procedure to transfer products from one supplier to another. If the old supplier or new supplier does not exist, rollback.

ANSWER:

```

CREATE OR REPLACE PROCEDURE transferProductSupplier(
    newOldSupplierId INT,
    newNewSupplierId INT,
    newProductId INT
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM suppliers WHERE supplierid = newOldSupplierId
    ) THEN
        RAISE EXCEPTION 'Old supplier does not exist';
    END IF;

```

```
IF NOT EXISTS (  
    SELECT 1 FROM suppliers WHERE supplierid = newNewSupplierId  
) THEN  
    RAISE EXCEPTION 'New supplier does not exist';  
END IF;  
  
IF NOT EXISTS (  
    SELECT 1 FROM products  
    WHERE productid = newProductId  
    AND supplierid = newOldSupplierId  
) THEN  
    RAISE EXCEPTION 'Product does not belong to old supplier';  
END IF;  
  
UPDATE products  
SET supplierid = newNewSupplierId  
WHERE productid = newProductId;  
  
COMMIT;  
  
EXCEPTION  
    WHEN OTHERS THEN  
        ROLLBACK;  
        RAISE NOTICE 'Transaction failed: %', SQLERRM;  
END;  
$;
```

OUTPUT:

```
1 CREATE OR REPLACE PROCEDURE transferProductSupplier(  
2     newOldSupplierId INT,  
3     newNewSupplierId INT,  
4     newProductId INT  
5 )  
6 LANGUAGE plpgsql  
7 AS $$  
8 BEGIN  
9     IF NOT EXISTS (  
10        SELECT 1 FROM suppliers WHERE supplierid = newOldSupplierId  
11    ) THEN  
12        RAISE EXCEPTION 'Old supplier does not exist';  
13    END IF;  
14  
15     IF NOT EXISTS (  
16        SELECT 1 FROM suppliers WHERE supplierid = newNewSupplierId  
17    ) THEN  
18        RAISE EXCEPTION 'New supplier does not exist';  
19    END IF;  
20  
21    UPDATE products  
22    SET unitprice = unitprice * newNewSupplierId / newOldSupplierId  
23    WHERE productid = newProductId;  
24  
25    COMMIT;  
26 END;  
27
```

Data Output Messages Notifications

CALL

Query returned successfully in 989 msec.

6) Create a stored procedure to update the price of all products in a category by a percentage. Rollback if the percentage is less than or equal to zero.

ANSWER:

```
CREATE OR REPLACE PROCEDURE updateCategoryPrice(  
    newCategoryId INT,  
    newPercentage NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF newPercentage <= 0 THEN  
        RAISE EXCEPTION 'Percentage must be greater than zero';  
    END IF;  
  
    IF NOT EXISTS (  
        SELECT 1 FROM categories WHERE categoryid = newCategoryId  
    ) THEN  
        RAISE EXCEPTION 'Category does not exist';  
    END IF;  
  
    UPDATE products  
    SET unitprice = unitprice + (unitprice * newPercentage / 100)  
    WHERE categoryid = newCategoryId;
```

COMMIT;

EXCEPTION

WHEN OTHERS THEN

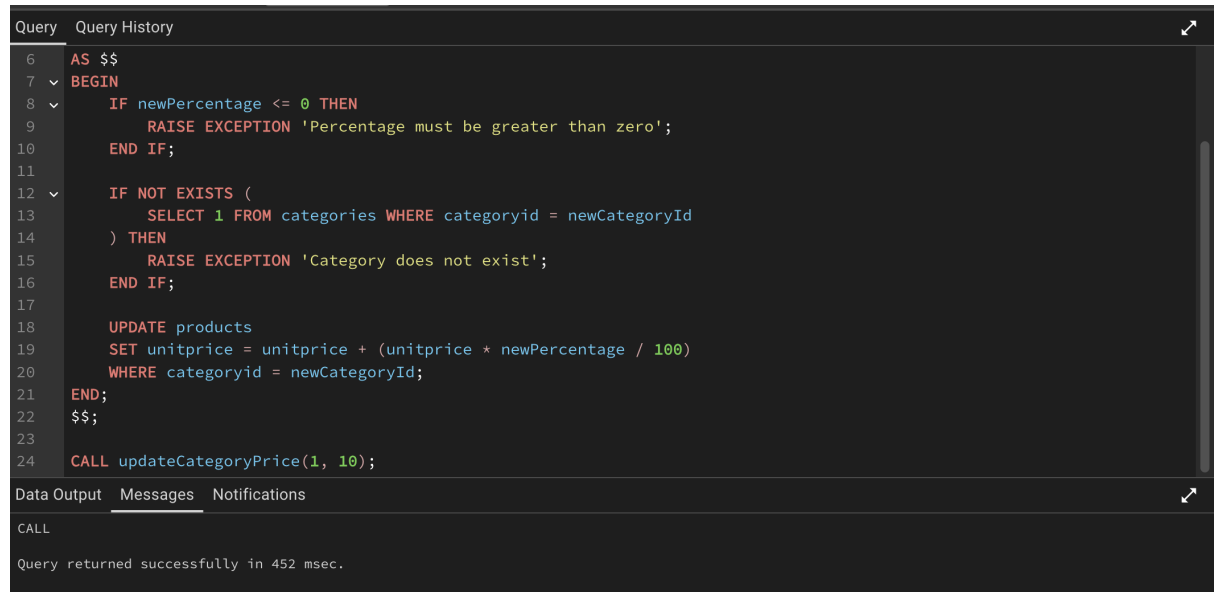
ROLLBACK;

RAISE NOTICE 'Transaction failed: %', SQLERRM;

END;

\$\$;

OUTPUT:



The screenshot shows a database query editor with a dark theme. The top panel, titled 'Query' and 'Query History', displays a PL/SQL procedure. The code is as follows:

```
6 AS $$
7 BEGIN
8     IF newPercentage <= 0 THEN
9         RAISE EXCEPTION 'Percentage must be greater than zero';
10    END IF;
11
12    IF NOT EXISTS (
13        SELECT 1 FROM categories WHERE categoryid = newCategoryId
14    ) THEN
15        RAISE EXCEPTION 'Category does not exist';
16    END IF;
17
18    UPDATE products
19    SET unitprice = unitprice + (unitprice * newPercentage / 100)
20    WHERE categoryid = newCategoryId;
21 END;
22 $$;
23
24 CALL updateCategoryPrice(1, 10);
```

The bottom panel, titled 'Data Output', 'Messages', and 'Notifications', shows the execution result:

```
CALL
Query returned successfully in 452 msec.
```

7) Create a stored procedure to add a new product under an existing category and supplier. Rollback if the category or supplier does not exist.

ANSWER:

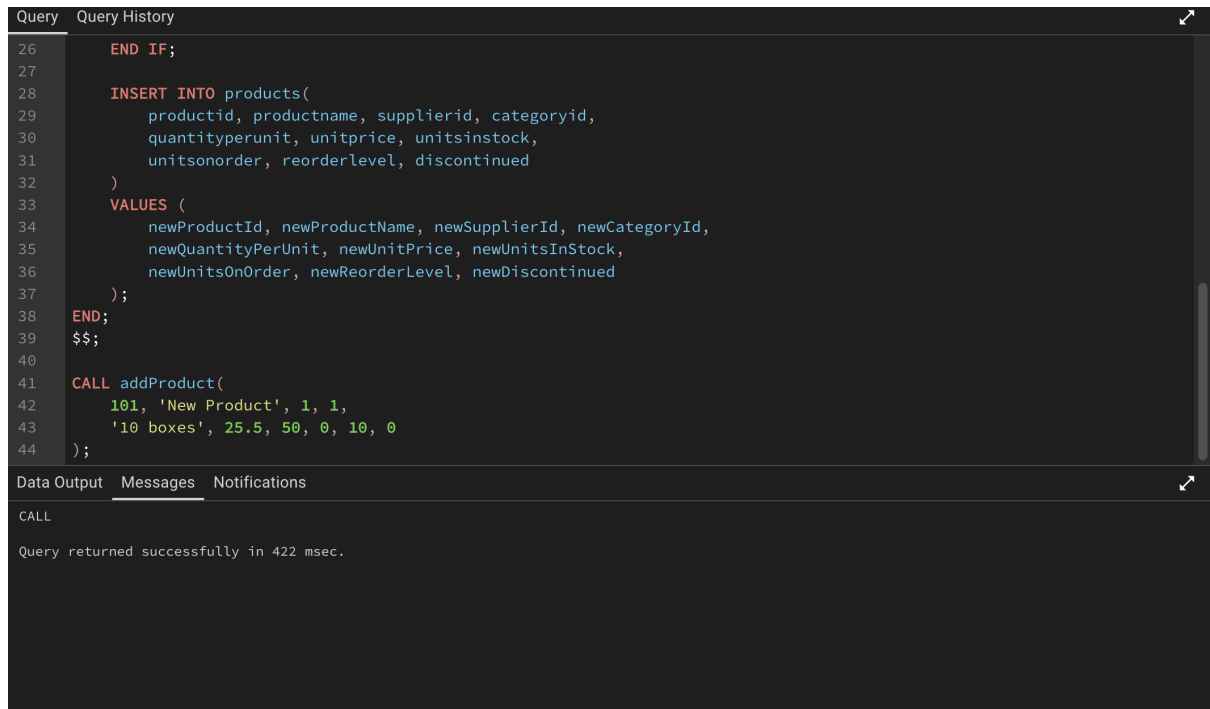
```
CREATE OR REPLACE PROCEDURE addProduct(
    newProductId INT,
    newProductName VARCHAR,
    newSupplierId INT,
    newCategoryId INT,
    newQuantityPerUnit VARCHAR,
    newUnitPrice NUMERIC,
    newUnitsInStock INT,
    newUnitsOnOrder INT,
    newReorderLevel INT,
    newDiscontinued INT
)
LANGUAGE plpgsql
AS $$
```

```
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM categories WHERE categoryid = newCategoryId
  ) THEN
    RAISE EXCEPTION 'Category does not exist';
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM suppliers WHERE supplierid = newSupplierId
  ) THEN
    RAISE EXCEPTION 'Supplier does not exist';
  END IF;

  INSERT INTO products(
    productid, productname, supplierid, categoryid,
    quantityperunit, unitprice, unitsinstock,
    unitsonorder, reorderlevel, discontinued
  )
  VALUES (
    newProductId, newProductName, newSupplierId, newCategoryId,
    newQuantityPerUnit, newUnitPrice, newUnitsInStock,
    newUnitsOnOrder, newReorderLevel, newDiscontinued
  );
END;
$$;
CALL addProduct(101, 'New Product', 1, 1, '10 boxes', 25.5, 50, 0, 10, 0);
```

OUTPUT:



```
Query    Query History
26      END IF;
27
28      INSERT INTO products(
29          productid, productname, supplierid, categoryid,
30          quantityperunit, unitprice, unitsinstock,
31          unitsonorder, reorderlevel, discontinued
32      )
33      VALUES (
34          newProductId, newProductName, newSupplierId, newCategoryId,
35          newQuantityPerUnit, newUnitPrice, newUnitsInStock,
36          newUnitsOnOrder, newReorderLevel, newDiscontinued
37      );
38  END;
39  $$;
40
41  CALL addProduct(
42      101, 'New Product', 1, 1,
43      '10 boxes', 25.5, 50, 0, 10, 0
44  );
```

Data Output Messages Notifications

CALL

Query returned successfully in 422 msec.

8) Create a stored procedure to delete a customer only if the customer has no orders. Use a transaction and rollback if orders exist.

ANSWER:

```
CREATE OR REPLACE PROCEDURE deleteCustomer(
    newCustomerId VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM orders WHERE customerid = newCustomerId
    ) THEN
        RAISE EXCEPTION 'Customer has orders';
    END IF;

    DELETE FROM customers
    WHERE customerid = newCustomerId;

    COMMIT;

EXCEPTION
```



```
WHEN OTHERS THEN
    ROLLBACK;
    RAISE NOTICE 'Transaction failed';
END;
$$;
```

OUTPUT:

```
1 CREATE OR REPLACE PROCEDURE deleteCustomer(
2     newCustomerId VARCHAR
3 )
4 LANGUAGE plpgsql
5 AS $$
6 BEGIN
7     IF EXISTS (
8         SELECT 1 FROM orders WHERE customerid = newCustomerId
9     ) THEN
10        RAISE EXCEPTION 'Customer has orders';
11    END IF;
12
13    DELETE FROM customers
14    WHERE customerid = newCustomerId;
15
16 END;
17 $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 564 msec.

9) Create a stored procedure to apply a discount to all order details for a specific order. Rollback if the order does not exist.

ANSWER:

```
CREATE OR REPLACE PROCEDURE applyOrderDiscount(
    newOrderId INT,
    newDiscount FLOAT
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM orders WHERE orderid = newOrderId
    ) THEN
        RAISE EXCEPTION 'Order does not exist';
    END IF;

    UPDATE order_details
    SET discount = newDiscount
```

```

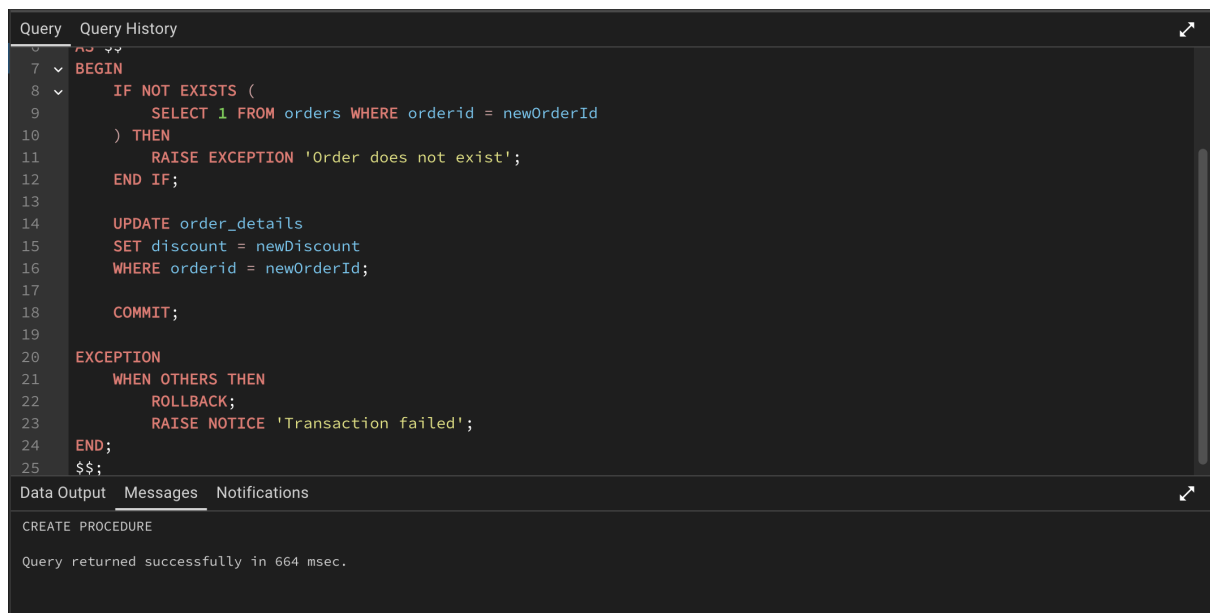
WHERE orderid = newOrderId;

COMMIT;

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE NOTICE 'Transaction failed';
END;
$$;

```

OUTPUT:



```

Query  Query History
7 BEGIN
8   IF NOT EXISTS (
9     SELECT 1 FROM orders WHERE orderid = newOrderId
10  ) THEN
11    RAISE EXCEPTION 'Order does not exist';
12  END IF;
13
14  UPDATE order_details
15  SET discount = newDiscount
16  WHERE orderid = newOrderId;
17
18  COMMIT;
19
20 EXCEPTION
21   WHEN OTHERS THEN
22     ROLLBACK;
23     RAISE NOTICE 'Transaction failed';
24 END;
25 $$;

```

CREATE PROCEDURE

Query returned successfully in 664 msec.

10) Create a stored procedure to place an order with multiple products. Insert the order and all order items in one transaction. If any product is invalid or stock is insufficient, rollback the complete order.

ANSWER:

```

CREATE OR REPLACE PROCEDURE placeOrderMultiple(
  newOrderId INT,
  newCustomerId VARCHAR,
  newEmployeeId INT,
  newProductIds INT[],
  newQuantities INT[],
  newDiscounts FLOAT[]
)
LANGUAGE plpgsql

```

```

AS $$
DECLARE
    i INT;
    v_price NUMERIC;
    v_stock INT;
BEGIN
    IF array_length(newProductIds, 1) IS NULL THEN
        RAISE EXCEPTION 'No products provided';
    END IF;

    IF NOT EXISTS (
        SELECT 1 FROM customers WHERE customerid = newCustomerId
    ) THEN
        RAISE EXCEPTION 'Customer does not exist';
    END IF;

    INSERT INTO orders(orderid, customerid, employeeid, orderdate)
    VALUES (newOrderId, newCustomerId, newEmployeeId, CURRENT_DATE);

    FOR i IN 1..array_length(newProductIds, 1) LOOP

        SELECT unitprice, unitsinstock
        INTO v_price, v_stock
        FROM products
        WHERE productid = newProductIds[i]
        FOR UPDATE;

        IF v_price IS NULL THEN
            RAISE EXCEPTION 'Invalid product id: %', newProductIds[i];
        END IF;

        IF v_stock < newQuantities[i] THEN
            RAISE EXCEPTION 'Insufficient stock for product %', newProductIds[i];
        END IF;

        INSERT INTO order_details(orderid, productid, unitprice, quantity, discount)
        VALUES (newOrderId, newProductIds[i], v_price, newQuantities[i],
newDiscounts[i]);

        UPDATE products
        SET unitsinstock = unitsinstock - newQuantities[i]

```

```
        WHERE productid = newProductIds[i];

    END LOOP;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE NOTICE 'Transaction failed: %', SQLERRM;
END;
$$;
```

OUTPUT:

```
43      END IF;
44
45      INSERT INTO order_details(orderid, productid, unitprice, quantity, discount)
46      VALUES (newOrderId, newProductIds[i], v_price, newQuantities[i], newDiscounts[i]);
47
48      UPDATE products
49      SET unitsinstock = unitsinstock - newQuantities[i]
50      WHERE productid = newProductIds[i];
51
52  END LOOP;
53
54  COMMIT;
55
56  EXCEPTION
57      WHEN OTHERS THEN
58          ROLLBACK;
59          RAISE NOTICE 'Transaction failed: %', SQLERRM;
60  END;
61  $$;
```

Data Output Messages Notifications

CREATE PROCEDURE

Query returned successfully in 1 secs 856 msec.